



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий
(МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Тестирование и верификация программного обеспечения»

Практическое занятие № 2

ИКБО-43-23 **ОГАНЕСЯН Д. К.**
КОЩЕЕВ М.И.
ГУТКОВИЧ А.А.
ИВАНОВ А.В.
УЛЗЕТУЕВ А.С.

(подпись)

АССИСТЕНТ

ИЛЬИЧЕВ Г.П.

(подпись)

**ОТЧЕТ
ПРЕДСТАВЛЕН**

«17»__ОКТЯБРЯ_2025 Г.

Москва 2025 г.

СОДЕРЖАНИЕ

1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ.....	3
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	4
2.1 Разработка модулей.....	4
2.2 Модульное тестирование.....	5
2.3 Мутационное тестирование.....	9
3 АНАЛИЗ И ВЫВОДЫ.....	15
3.1 Оценка качества тестирования.....	15
3.2 Анализ недостатков и их устранение.....	15
3.3 Итоговые выводы по результатам работы.....	16

1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ

Цель работы: познакомить студентов с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

Для достижения поставленной цели работы студентам необходимо выполнить ряд задач:

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Разработка модулей

Описание функциональности: Модуль представляет собой систему ведения истории конвертаций, которая позволяет загружать и сохранять данные в JSON-файл, добавлять новые записи с указанием категории, исходных и конечных единиц измерения, результата и временной метки, получать последние конвертации, фильтровать их по категориям, очищать всю историю и формировать статистику использования (общее количество конвертаций, распределение по категориям, даты первой и последней операции). Все операции с данными осуществляются через работу с файлом `conversion_history.json`, при этом предусмотрена возможность включения режима отладки для дополнительной информации о выполняемых операциях.

Листинг 2 — Исходный код.

```
import json
import datetime
from pathlib import Path

debug = False

FILENAME = Path("conversion_history.json")

def load_history():
    #Загрузить историю из файла
    if FILENAME.exists():
        with open(FILENAME, 'r', encoding='utf-8') as f:
            return json.load(f)
    return []

def save_history(history):
    #Сохранить историю в файл
    with open(FILENAME, 'w', encoding='utf-8') as f:
        json.dump(history, f, ensure_ascii=False, indent=2)
        print("Json has been saved.")
    if debug:
        print("-----")
        print(f"history: {history}")
        print("-----")

def add_conversion(category, value, from_unit, to_unit, result):
    #Добавить запись о конвертации
    history = load_history()
    entry = {
        "timestamp": datetime.datetime.now().isoformat(),
        "category": category,
        "value": value,
```

```

        "from_unit": from_unit,
        "to_unit": to_unit,
        "result": result
    }
    history.append(entry)
    save_history(history)

def get_recent_conversions(count=5):
    #Получить последние сколько-то конвертаций
    history = load_history()
    return history[count:]

def clear_history():
    #Очистить всю историю
    save_history([])

def get_conversions_by_category(category):
    #Получить все конвертации по категории
    history = load_history()
    return [entry for entry in history if entry["category"] == category]

def get_statistics():
    #Получить статистику по истории
    history = load_history()
    if not history:
        return {"total": 0}

    categories = {}
    for entry in history:
        cat = entry["category"]
        categories[cat] = categories.get(cat, 0) + 1

    return {
        "total": len(history),
        "by_category": categories,
        "first_conversion": history[0]["timestamp"],
        "last_conversion": history[-1]["timestamp"]
    }
}

```

2.2 Модульное тестирование

Описание тестов: Тестовый набор проверяет функциональность модуля истории конвертаций, включая загрузку и сохранение данных, добавление записей, получение последних конвертаций, фильтрацию по категориям, очистку истории и формирование статистики. Особое внимание уделяется проверке корректности временных меток и сохранению данных в JSON-файл.

Листинг 8 — Код тестов.

```

import pytest
import json
import core.history as hm
from pathlib import Path

@pytest.fixture(autouse=True)
def clear_history_file(tmp_path, monkeypatch):

```

```

test_file = tmp_path / "test_history.json"
monkeypatch.setattr(hm, "FILENAME", test_file)

if hasattr(hm, "_history_cache"):
    hm._history_cache = []

if hasattr(hm, "clear_history"):
    try:
        hm.clear_history()
    except Exception:
        pass

yield

if test_file.exists():
    test_file.unlink()

def test_load_history_empty():
    assert hm.load_history() == []

def test_save_and_load_history():
    test_data = [{"test": "data"}]
    hm.save_history(test_data)
    loaded_data = hm.load_history()
    assert loaded_data == test_data

def test_add_conversion():
    hm.add_conversion("длина", 100, "см", "м", 1.0)
    history = hm.load_history()
    assert len(history) == 1
    assert history[0]["category"] == "длина"
    assert history[0]["value"] == 100
    assert history[0]["from_unit"] == "см"
    assert history[0]["to_unit"] == "м"
    assert history[0]["result"] == 1.0

def test_get_recent_conversions():
    for i in range(3):
        hm.add_conversion("длина", i, "м", "см", i * 100)
    recent = hm.get_recent_conversions(2)
    assert len(recent) == 2
    assert recent[0]["value"] == 1
    assert recent[1]["value"] == 2

def test_clear_history():
    hm.add_conversion("масса", 1000, "г", "кг", 1.0)
    hm.clear_history()
    assert hm.load_history() == []

def test_get_conversions_by_category():
    hm.add_conversion("длина", 100, "см", "м", 1.0)
    hm.add_conversion("масса", 1000, "г", "кг", 1.0)
    hm.add_conversion("длина", 2, "м", "км", 0.002)
    length_conv = hm.get_conversions_by_category("длина")
    assert len(length_conv) == 2
    for conv in length_conv:
        assert conv["category"] == "длина"

def test_get_statistics_empty():
    stats = hm.get_statistics()

```

```

    assert stats == {"total": 0}

def test_get_statistics_with_data():
    hm.add_conversion("длина", 100, "см", "м", 1.0)
    hm.add_conversion("масса", 1000, "г", "кг", 1.0)
    hm.add_conversion("длина", 2, "м", "км", 0.002)
    stats = hm.get_statistics()
    assert stats["total"] == 3
    assert stats["by_category"]["длина"] == 2
    assert stats["by_category"]["масса"] == 1

def test_timestamp_format():
    hm.add_conversion("длина", 100, "см", "м", 1.0)
    history = hm.load_history()
    timestamp = history[0]["timestamp"]
    assert "T" in timestamp
    assert ":" in timestamp

def test_file_persistence():
    hm.add_conversion("длина", 150, "см", "м", 1.5)
    assert hm.FILENAME.exists()
    with open(hm.FILENAME, 'r', encoding='utf-8') as f:
        data = json.load(f)
    assert len(data) == 1
    assert data[0]["value"] == 150

```

Методология: Тестирование реализовано с использованием pytest и включает автоматическую очистку тестового файла истории перед каждым тестом. Проверяется как базовая функциональность (загрузка/сохранение), так и сложные сценарии работы с историей конвертаций, включая проверку структуры записей и их корректности.

Анализ покрытия кода: Тесты охватывают все ключевые функции модуля истории: работу с файлами, манипуляции с записями, формирование статистики и временные метки. Проверяется как позитивный сценарий (корректная работа с данными), так и граничные случаи, включая пустую историю и различные категории конвертаций.

Исправление ошибок:

Краткое описание ошибки: «Неверное считывание недавней истории конверсии.».

Статус ошибки: открыта («Open»).

Категория ошибки: критическая («Critical»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Запустить тест:

```
for i in range(5):
```

```
    core.history.add_conversion("длина", i, "м", "см", i * 100)
```

```
recent = core.history.get_recent_conversions(2)
```

2. Полученный результат: «recent[0], recent[1], recent[2]». Ожидаемый результат: «recent[3], recent[4]».

Листинг 9 — Исправленный код.

```
import json
import datetime
from pathlib import Path
debug = False

FILENAME = Path("conversion_history.json")

def load_history():
    #Загрузить историю из файла
    if FILENAME.exists():
        with open(FILENAME, 'r', encoding='utf-8') as f:
            return json.load(f)
    return []

def save_history(history):
    #Сохранить историю в файл
    with open(FILENAME, 'w', encoding='utf-8') as f:
        json.dump(history, f, ensure_ascii=False, indent=2)
        print("Json has been saved.")
    if debug:
        print("-----")
        print(f"history: {history}")
        print("-----")

def add_conversion(category, value, from_unit, to_unit, result):
    #Добавить запись о конвертации
    history = load_history()
    entry = {
        "timestamp": datetime.datetime.now().isoformat(),
        "category": category,
        "value": value,
        "from_unit": from_unit,
        "to_unit": to_unit,
        "result": result
    }
    history.append(entry)
    save_history(history)

def get_recent_conversions(count=5):
    #Получить последние сколько-то конвертаций
```



```

history = load_history()
return history[-count:]

def clear_history():
    #Очистить всю историю
    save_history([])

def get_conversions_by_category(category):
    #Получить все конвертации по категории
    history = load_history()
    return [entry for entry in history if entry["category"] == category]

def get_statistics():
    #Получить статистику по истории
    history = load_history()
    if not history:
        return {"total": 0}

    categories = {}
    for entry in history:
        cat = entry["category"]
        categories[cat] = categories.get(cat, 0) + 1

    return {
        "total": len(history),
        "by_category": categories,
        "first_conversion": history[0]["timestamp"],
        "last_conversion": history[-1]["timestamp"]
    }

```

2.3 Мутационное тестирование

Создание мутантов:

1. Мутации в load_history

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
if FILENAME.exists():	if not FILENAME.exists(): (Инверсия условия)	test_load_history_empty Провалится (файл не существует, но функция попытается его прочитать)	 Да	Нет
return []	return [1] (Изменение)	test_load_history	 Да	Нет

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
	возвращаемого значения по умолчанию)	у_empty Провалится (ожидается []), возвращается [1])		

2. Мутации в save_history

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
print("Json has been saved.")	Удаление строки (Удаление оператора)	Выживет	✗ Нет	Да
with open(FILENAME, 'w', encoding='utf-8')	with open(FILENAME, 'a', encoding='utf-8') (Замена режима 'w' на 'a')	test_save_and_load_history Выживет (При первом запуске тестов файл пуст, 'w' и 'a' ведут себя одинаково)	✗ Нет	Да

3. Мутации в add_conversion и get_recent_conversions

Функция	Исходный код	Мутант (Ошибка)	Результат теста
add_conversion	history.append(entry)	Удаление строки (Удаление оператора)	test_add_conversion Провалится (len(history) будет 0)
add_conversion	"timestamp":	"timestamp	test_time

Функция	Исходный код	Мутант (Ошибка)	Результат теста
rsion	<code>datetime.datetime.now().isoformat(),</code>	<code>p": "123",</code> (Изменение логики поля)	<code>stamp_format</code> Провалится ("Т" или ":" не будет в "123")
<code>get_recent_conversions</code>	<code>return history[-count:]</code>	<code>return history[:count]</code> (Изменение среза)	<code>test_get_recent_conversions</code> Провалится (вернет первые 2 записи, а не последние)
<code>get_recent_conversions</code>	<code>return history[-count:]</code>	<code>return history</code> (Удаление среза)	<code>test_get_recent_conversions</code> Провалится (вернет 3 записи вместо 2)

4. Мутации в `get_conversions_by_category`

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
<code>entry["category"] == category</code>	<code>entry["category"] != category</code> (Инверсия сравнения)	<code>test_get_conversions_by_category</code> Провалится (длина будет 1 (масса), а не 2 (длина))	 Да	Нет
<code>entry["category"] == category</code>	<code>entry["value"] == category</code> (Изменение)	<code>test_get_conversions_by_category</code> Провалится	 Да	Нет

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
	ключа)	(длина будет 0, т.к. 100 != "длина")		

5. Мутации в get_statistics

Исходный код	Мутант (Ошибка)	Результат теста	Убит ?	Недостаток теста?
if not history:	if history: (Инверсия условия)	test_get_statistics_empty Провалится (будет ошибка IndexError при попытке получить history[0])	 Да	Нет
categories.get(cat, 0) + 1	categories.get(cat, 0) + 2 (Изменение приращения)	test_get_statistics_with_data Провалится (счетчик будет 3 вместо 2)	 Да	Нет
history[0]["timestamp"]	history[-1]["timestamp"] (Изменение индекса)	test_get_statistics_with_data Выживет	 Нет	Да

Анализ их выживания:

Мутант 1 (Удаление print): Тесты не проверяют вывод в консоль, что является ожидаемым поведением, поскольку это не влияет на функциональную логику. Мутант выживает, но это эквивалентный мутант или мутант, не

имеющий отношения к бизнес-логике.

Мутант 2 (Режим записи): Если заменить 'w' (перезапись) на 'a' (добавление), тесты, которые добавляют записи (например, `test_add_conversion`), будут работать неправильно, если запускались бы без очистки между собой. Однако `test_save_and_load_history` выживает, потому что файл пуст.

Мутант 3 (Статистика): Мутант, который менял `history[0]` на `history[-1]` при получении `first_conversion`, выжил. Это случилось потому, что наш тест `test_get_statistics_with_data` не проверял фактические значения `first_conversion` и `last_conversion`, а фокусировался только на общих подсчетах.

Корректировка тестов:

1. Чтобы убить выжившего мутанта в `get_statistics`, мы добавили явную проверку того, что `first_conversion` и `last_conversion` действительно содержат разные значения.

Листинг 18 — Код теста.

```
def test_get_statistics_timestamps(self):
    # Добавляем две конвертации, чтобы убедиться, что они имеют разное время
    hm.add_conversion("длина", 100, "см", "м", 1.0)
    # Искусственная задержка для гарантированно разного timestamp
    import time; time.sleep(0.01)
    hm.add_conversion("масса", 1000, "г", "кг", 1.0)

    stats = hm.get_statistics()

    # 1. Проверяем, что первый и последний timestamp разные
    assert stats["first_conversion"] != stats["last_conversion"]

    # 2. Проверяем, что первый - это первый, а последний - это последний
    (опционально, но желательно)
    history = hm.load_history()
    assert stats["first_conversion"] == history[0]["timestamp"]
    assert stats["last_conversion"] == history[-1]["timestamp"]
```

2. Чтобы убить выжившего мутанта в `save_history` (Режим 'w' vs 'a'), мы убедились, что один тест сохраняет данные, а следующий загружает их и обнаруживает, что старые данные были удалены (т.е. что произошла перезапись).

Листинг 19 — Код теста.

```
def test_save_history_overwrites(self):
    # 1. Сохраняем первую запись
    hm.save_history([{"id": 1}])
    # 2. Сохраняем вторую запись (должна перезаписать)
    hm.save_history([{"id": 2}])

    loaded_data = hm.load_history()
    # Если бы был режим 'a', то len было бы 2. В режиме 'w' должно быть 1.
    assert len(loaded_data) == 1
    assert loaded_data[0]["id"] == 2
```

3 АНАЛИЗ И ВЫВОДЫ

3.1 Оценка качества тестирования

Итоговое Качество Тестирования:

1. Полнота (Покрытие): Теперь тесты обеспечивают не только высокое покрытие строк кода, но и высокое покрытие логики. Почти все значимые операторы (математические, логические, сравнения) проверяются на их чувствительность к ошибкам.
2. Надежность (Чувствительность): Набор тестов способен обнаруживать даже атомарные, синтаксически небольшие ошибки (мутанты), которые могли бы быть допущены программистом, что является прямым доказательством его надежности.
3. Тестирование интеграционных точек: Особое внимание уделено интеграционным точкам (например, view вызывает validator и history). Проверка, что удаление валидации приводит к сбою, а удаление записи в историю обнаруживается, делает интеграционные тесты более надежными.

3.2 Анализ недостатков и их устранение

Улучшения, которые были достигнуты благодаря мутационному тестированию:

Модуль	Ранее Выжившие Мутанты (Пробелы)	Статус после изменений
history	Отсутствовала проверка перезаписи файла ('w' vs 'a') и проверки временных меток в статистике.	Убиты. Добавлены тесты, гарантирующие, что файл перезаписывается при сохранении, и что поля <code>first_conversion</code> и <code>last_conversion</code> в статистике различаются и корректны.

3.3 Итоговые выводы по результатам работы

В результате выполнения практической работы были достигнуты поставленные цели: освоены методы модульного и мутационного тестирования, разработаны и проанализированы тестовые наборы. В ходе исследования успешно применено мутационное тестирование, созданы эффективные модульные тесты с высоким покрытием кода. Мутационное тестирование позволило выявить и устранить пробелы в тестовом наборе, повысив качество проверки программного продукта. Приобретены практические навыки разработки тестов, анализа покрытия кода и оценки эффективности тестовых сценариев. Полученные результаты подтверждают, что комбинированный подход к тестированию обеспечивает надёжную проверку качества программного обеспечения и помогает обнаруживать потенциальные ошибки на ранних этапах разработки.